Table des matières

Labo 01 — Analyser les sources 1
Avant de commencer 2
Étape 1 — Créer le dépôt 01_de 2
Étape 2 — Lire la documentation et vérifier la livraison 3
Étape 3 — Profiler chaque source 4
Étape 4 — Mesurer ce que coûte un type deviné 10
Étape 5 — Sauvegarder les requêtes dans le dépôt 12
Dépannage 13

Labo 01 — Analyser les sources

Labo · Cours 1/12 — Formation Data Engineering

But : créer le dépôt `01_de`, puis faire le premier regard d'un ingénieur de données sur tout ce que Boréal a livré : lire la documentation, vérifier la livraison, profiler chaque source, repérer les pièges des types devinés et garder les requêtes dans le dépôt. Le pourquoi de chaque étape est dans `Theorie_01_DE.md`, section 8 et annexe A.

Deux outils :

- l'Anaconda Prompt, environnement `formations` actif : Git et les commandes du système ;
- l'interface web de DuckDB (`duckdb -ui`), un notebook de requêtes dans le navigateur : le SQL.

Liste de contrôle :

- ☐ Dépôt `01_de` créé, `.gitignore` validé, `git status` propre
- ☐ Documentation lue : les sept `LISEZMOI.md`, le dictionnaire sous la main
- ☐ Livraison vérifiée : 1 763 fichiers attendus, 1 763 présents, 0 altéré
- ☐ Six sources profilées : usine, ERP, garanties, télémétrie, météo, corpus
- ☐ Liens vérifiés : numéros de série et concessionnaires, aucun orphelin

© 2026 iAlexMG. Tous droits réservés.

- ☐ Pièges des types vus : `20251232`, un décalage perdu, une colonne `JSON`, des dates triées en texte, une jointure qui plante
- ☐ Requêtes sauvegardées dans `01_de\sql\premier_regard.sql`, rejouées par `duckdb -f`, validées dans Git

---

Avant de commencer

Le projet et ses sources viennent du Cours 00, `Cours_00_DE_Boreal.md` : s'il n'est pas fait, le faire d'abord. Anaconda Prompt, environnement `formations` actif, placée dans `Projet_Boreal_intermediaire` :

```
dir sources
REM Attendu : six dossiers, corpus, erp, garanties, meteo, telemetrie et usine.
```

L'Anaconda Prompt reste dans `Projet_Boreal_intermediaire` pour tout le labo : aucun `cd` à taper ensuite.

Lire `02_Boreal_intermediaire/Mise_en_situation_Boreal_01_DE.md`.

Étape 1 — Créer le dépôt `01_de`

Théorie : `Theorie_01_DE.md`, annexe A.1.

Anaconda Prompt, depuis `Projet_Boreal_intermediaire` :

```
git init 01_de
REM Attendu : Initialized empty Git repository in …/Projet_Boreal_intermediaire/01_de/.git/
notepad 01_de\.gitignore
```

Le Bloc-notes propose de créer le fichier : accepter. Coller, enregistrer, fermer :

```
# Secrets (mots de passe, jetons) : jamais dans Git
.env

# Fichiers compilés par Python
__pycache__/
```

Valider, sans quitter `Projet_Boreal_intermediaire` : `git -C 01_de` lance Git dans le dépôt, et les chemins qui suivent partent de `01_de`.

```
git -C 01_de add .gitignore
git -C 01_de commit -m "Cours 01 : le dépôt 01_de et son .gitignore"
git -C 01_de status
REM Attendu : nothing to commit, working tree clean.
```


© 2026 iAlexMG. Tous droits réservés.

L'avertissement `LF will be replaced by CRLF` est sans conséquence.

Étape 2 — Lire la documentation et vérifier la livraison

Théorie : `Theorie_01_DE.md`, annexe A.2.

2.1 Lire la documentation livrée avec les sources, dans l'éditeur de votre choix (Bloc-notes, PyCharm), dans cet ordre :

1. `sources/LISEZMOI.md`
2. `sources/usine/LISEZMOI.md`
3. `sources/erp/LISEZMOI.md`
4. `sources/garanties/LISEZMOI.md`
5. `sources/telemetrie/LISEZMOI.md`
6. `sources/meteo/LISEZMOI.md`
7. `sources/corpus/LISEZMOI.md`

Garder `sources/dictionnaire_boreal.md` ouvert pour la suite : le sens de chaque colonne.

Lancer l'interface. Anaconda Prompt, depuis `Projet_Boreal_intermediaire` :

```
duckdb -ui
```

- L'interface s'ouvre dans le navigateur, à `http://localhost:4213`. Au premier lancement, DuckDB télécharge son extension `ui` : il faut Internet.
- Laisser l'Anaconda Prompt ouvert : le fermer arrête l'interface.
- Créer un notebook (`+` à côté de Notebooks) et le renommer « Labo 01 » (double-clic sur son titre, puis Entrée) : chaque labo aura le sien.
- Une requête par cellule (`Add Cell`). Ctrl+Entrée l'exécute. Une cellule qui contient plusieurs requêtes n'affiche que le dernier résultat.
- Les lignes `--` donnent le résultat attendu ; DuckDB les ignore. L'interface n'a pas de cellule de texte : un titre s'écrit aussi en `--`, en tête de la cellule de sa requête.

2.2 Faire l'inventaire de tout ce qui est arrivé.

```
-- Attendu : 9 lignes, les 6 dossiers et 3 fichiers de la racine. telemetrie : 885 fichiers,
-- 70 571 707 octets, 41 vides ; usine : 144 fichiers ; erp : 452 fichiers. Aucun autre fichier vide.
SELECT split_part(replace(filename, '\', '/'), '/', 2) AS dossier,
       COUNT(*) AS fichiers, SUM(size) AS octets, COUNT(*) FILTER (WHERE size = 0) AS vides
```


© 2026 iAlexMG. Tous droits réservés.

```
FROM read_blob('sources/**')
GROUP BY ALL
ORDER BY dossier;
```

2.3 Détailler l'usine et l'ERP.

```
-- Attendu : 8 lignes. usine/historien : 92 fichiers, 13 725 380 octets ; usine/compteur : 48 ;
-- usine/gmao : 1 ; usine/referentiel : 2 ; erp/extraits : 441 ; erp/postgresql : 10 ; 2 LISEZMOI.md.
SELECT split_part(replace(filename, '\', '/'), '/', 2) || '/' ||
       split_part(replace(filename, '\', '/'), '/', 3) AS dossier,
       COUNT(*) AS fichiers, SUM(size) AS octets
FROM read_blob(['sources/usine/**', 'sources/erp/**'])
GROUP BY ALL
ORDER BY dossier;
```

Noter deux tailles : 13,7 Mo d'historien, 70,6 Mo de télémétrie. Au Cours 02, les mêmes données converties en Parquet y seront comparées.

2.4 Vérifier l'intégrité de la livraison.

```
-- Attendu : 1 763 attendus, 1 763 présents, 0 altéré.
WITH attendu AS (
  SELECT empreinte, fichier
  FROM read_csv('sources/empreintes.sha256', delim = ' ', header = false,
                columns = {'empreinte': 'VARCHAR', 'vide': 'VARCHAR', 'fichier': 'VARCHAR'})
),
recu AS (
  SELECT sha256(content) AS empreinte, substr(replace(filename, '\', '/'), 9) AS fichier
  FROM read_blob('sources/**')
)
SELECT COUNT(*) AS attendus, COUNT(r.fichier) AS presents,
       COUNT(*) FILTER (WHERE r.empreinte <> a.empreinte) AS alteres
FROM attendu a
LEFT JOIN recu r USING (fichier);
```

Étape 3 — Profiler chaque source

Théorie : `Theorie_01_DE.md`, annexe A.3.

Toujours dans le même notebook, une requête par cellule.

L'usine : l'historien

3.1 Compter les lignes de l'historien.

© 2026 iAlexMG. Tous droits réservés.

```sql
-- Attendu : 275 436.
SELECT COUNT(*) AS lignes
FROM 'sources/usine/historien/*.csv';
```

3.2 Voir les types devinés.

```sql
-- Attendu : 4 lignes. TagName VARCHAR, Timestamp TIMESTAMP, Value DOUBLE, Quality BIGINT.
DESCRIBE SELECT * FROM 'sources/usine/historien/*.csv';
```

3.3 Répartir les codes de qualité.

```sql
-- Attendu : 192 (275 080 lignes), 64 (35), 0 (321).
SELECT Quality, COUNT(*) AS points
FROM 'sources/usine/historien/*.csv'
GROUP BY Quality
ORDER BY Quality DESC;
```

3.4 Compter les points par étiquette.

```sql
-- Attendu : 58 lignes. En tête, ESS.BANC.TEMP_REFROIDISSEMENT (27 661) ; en queue, UTL.CMP3.MARCHE (210).
SELECT TagName, COUNT(*) AS points
FROM 'sources/usine/historien/*.csv'
GROUP BY TagName
ORDER BY points DESC;
```

3.5 Lire les premiers points d’une étiquette.

```sql
-- Attendu : 10 lignes, du 2025-09-30 23:36:00 au 2025-10-01 06:06:00.
SELECT Timestamp, Value, Quality
FROM 'sources/usine/historien/*.csv'
WHERE TagName = 'UTL.CMP2.VIBRATION'
ORDER BY Timestamp
LIMIT 10;
```

3.6 Compter les lignes fichier par fichier.

```sql
-- Attendu : 92 lignes, de 1 599 (historien_2025-10-05.csv) à 4 612 (historien_2025-10-24.csv).
SELECT parse_filename(filename) AS fichier, COUNT(*) AS lignes
FROM read_csv('sources/usine/historien/*.csv', filename = true)
GROUP BY ALL
ORDER BY lignes;
```

3.7 Comparer les lignes aux points distincts.

© 2026 iAlexMG. Tous droits réservés.

```sql
-- Attendu : 275 436 lignes, 270 109 points distincts.
SELECT COUNT(*) AS lignes, COUNT(DISTINCT (TagName, Timestamp)) AS points_distincts
FROM 'sources/usine/historien/*.csv';
```

L'usine : le référentiel, la GMAO et le compteur

3.8 Vérifier que chaque étiquette de l'historien existe au référentiel.

```sql
-- Attendu : 58 au référentiel, 58 dans l'historien, 0 absente.
SELECT (SELECT COUNT(*) FROM 'sources/usine/referentiel/etiquettes.csv') AS au_referentiel,
       COUNT(DISTINCT h.TagName) AS dans_historien,
       COUNT(DISTINCT h.TagName) FILTER (WHERE e.etiquette IS NULL) AS absentes
FROM 'sources/usine/historien/*.csv' h
LEFT JOIN 'sources/usine/referentiel/etiquettes.csv' e ON e.etiquette = h.TagName;
```

3.9 Profiler les bons de travail de la GMAO.

```sql
-- Attendu : 779 bons, 779 numéros distincts, 361 correctifs, 2 encore ouverts.
SELECT COUNT(*) AS bons, COUNT(DISTINCT no_bt) AS numeros,
       COUNT(*) FILTER (WHERE type_bt = 'CORR') AS correctifs,
       COUNT(*) FILTER (WHERE ferme_le IS NULL) AS ouverts
FROM 'sources/usine/gmao/bons_travail.csv';
```

3.10 Vérifier que chaque actif de la GMAO existe au référentiel.

```sql
-- Attendu : 17 actifs cités, 0 absent.
SELECT COUNT(DISTINCT b.actif_id) AS actifs_cites,
       COUNT(DISTINCT b.actif_id) FILTER (WHERE a.actif_id IS NULL) AS absents
FROM 'sources/usine/gmao/bons_travail.csv' b
LEFT JOIN 'sources/usine/referentiel/actifs.csv' a USING (actif_id);
```

3.11 Profiler les relevés du compteur électrique.

```sql
-- Attendu : 140 255 relevés, 140 255 instants distincts, du 2022-01-01 00:15 au 2025-12-31 23:45
-- (heure de l'Est), 239 estimés.
SELECT COUNT(*) AS releves, COUNT(DISTINCT fin_intervalle) AS instants,
       MIN(fin_intervalle) AS premier, MAX(fin_intervalle) AS dernier,
       COUNT(*) FILTER (WHERE qualite = 'E') AS estimes
FROM 'sources/usine/compteur/*.csv';
```

L'ERP

3.12 Compter les lignes de chaque table, tous extraits confondus.

© 2026 iAlexMG. Tous droits réservés.

```sql
-- Attendu : 9 lignes. CDHDR et CDPOS : 5 extraits, 53 lignes chacune ; les 7 autres tables :
-- 49 extraits. VBAK 26 538 lignes, VBAP 81 292, ZSERIE 32 597, KNA1 7 481.
SELECT regexp_extract(replace(filename, '\', '/'), '([A-Z0-9]+)\.csv$', 1) AS table_sap,
       COUNT(DISTINCT filename) AS extraits, COUNT(*) AS lignes
FROM read_csv('sources/erp/extraits/*/*.csv', union_by_name = true, all_varchar = true, filename = true)
GROUP BY ALL
ORDER BY table_sap;
```

3.13 Comparer les lignes de VBAK (les en-têtes de commande) aux commandes distinctes.

```sql
-- Attendu : 26 538 lignes, 25 483 commandes distinctes.
SELECT COUNT(*) AS lignes, COUNT(DISTINCT VBELN) AS commandes
FROM 'sources/erp/extraits/*/VBAK.csv';
```

3.14 Repérer l'extrait livré deux fois.

```sql
-- Attendu : 609 en-têtes dans chacun.
SELECT '2023-03' AS extrait, COUNT(*) AS en_tetes FROM 'sources/erp/extraits/2023-03/VBAK.csv'
UNION ALL
SELECT '2023-03_bis', COUNT(*) FROM 'sources/erp/extraits/2023-03_bis/VBAK.csv';
```

3.15 Chercher les orphelins : des postes sans en-tête, des commandes d'un client inconnu.

```sql
-- Attendu : 0 et 0.
SELECT
  (SELECT COUNT(DISTINCT VBELN) FROM read_csv('sources/erp/extraits/*/VBAP.csv', all_varchar = true)
   WHERE VBELN NOT IN (SELECT VBELN FROM read_csv('sources/erp/extraits/*/VBAK.csv', all_varchar = true)))
   AS postes_sans_entete,
  (SELECT COUNT(DISTINCT KUNNR) FROM read_csv('sources/erp/extraits/*/VBAK.csv', all_varchar = true)
   WHERE KUNNR NOT IN (SELECT KUNNR FROM read_csv('sources/erp/extraits/*/KNA1.csv', all_varchar = true)))
   AS clients_hors_kna1;
```

Les garanties

3.16 Compter les lignes et les identifiants de chaque fichier, tous mois confondus.

```sql
-- Attendu : 5 lignes. clients 25 379 lignes pour 23 527 clients ; enregistrements 26 732 et 26 732 ;
-- visites 53 865 et 53 865 ; consentements 32 708 pour 23 527 clients ; reclamations 5 623 pour 3 348.
SELECT 'clients' AS fichier, COUNT(*) AS lignes, COUNT(DISTINCT client_id) AS identifiants
FROM 'sources/garanties/*/clients.csv'
UNION ALL SELECT 'enregistrements', COUNT(*), COUNT(DISTINCT no_enregistrement)
FROM 'sources/garanties/*/enregistrements.csv'
UNION ALL SELECT 'visites', COUNT(*), COUNT(DISTINCT no_visite)
FROM 'sources/garanties/*/visites.csv'
```


© 2026 iAlexMG. Tous droits réservés.

```sql
UNION ALL SELECT 'consentements', COUNT(*), COUNT(DISTINCT client_id)
FROM 'sources/garanties/*/consentements.csv'
UNION ALL SELECT 'reclamations', COUNT(*), COUNT(DISTINCT reclamation_id)
FROM read_json('sources/garanties/*/reclamations.json');
```

3.17 Suivre un client effacé (Loi 25) d'un extrait à l'autre, sans afficher ses données.

```sql
-- Attendu : 2 lignes pour CL0011163. Extrait 2024-01 : ACTIF, nom présent ; extrait 2025-09 :
-- EFFACE, nom absent.
SELECT client_id, regexp_extract(replace(filename, '\', '/'), 'garanties/([^/]+)/', 1) AS extrait,
       statut, nom IS NOT NULL AS nom_present
FROM read_csv('sources/garanties/*/clients.csv', filename = true)
WHERE client_id IN (SELECT client_id FROM 'sources/garanties/*/clients.csv' WHERE statut = 'EFFACE')
ORDER BY extrait;
```

La télémétrie

3.18 Compter les messages.

```sql
-- Attendu : 158 146.
SELECT COUNT(*) AS messages
FROM 'sources/telemetrie/*.jsonl';
```

3.19 Compter les messages en double.

```sql
-- Attendu : 158 146 messages, 155 843 distincts, 2 303 en double.
SELECT COUNT(*) AS messages, COUNT(DISTINCT message.message_id) AS distincts,
       COUNT(*) - COUNT(DISTINCT message.message_id) AS en_double
FROM read_json('sources/telemetrie/*.jsonl', union_by_name = true);
```

3.20 Voir le format changer avec le micrologiciel.

```sql
-- Attendu : 3 lignes. 1.3.0 (71 029 messages) et 1.6.2 (17 649) : temp_moteur_max seulement ;
-- 2.0.1 (69 468) : temperature_moteur_max et heures_moteur seulement.
SELECT message.micrologiciel AS micrologiciel, COUNT(*) AS messages,
       COUNT(message.temp_moteur_max) AS temp_moteur_max,
       COUNT(message.temperature_moteur_max) AS temperature_moteur_max,
       COUNT(message.heures_moteur) AS heures_moteur
FROM read_json('sources/telemetrie/*.jsonl', union_by_name = true)
GROUP BY ALL
ORDER BY micrologiciel;
```

3.21 Mesurer les messages en retard.

© 2026 iAlexMG. Tous droits réservés.

```sql
-- Attendu : 291 messages reçus plus d'un jour après la fin de leur sortie ; le pire, 4 jours et 33 secondes.
SELECT COUNT(*) FILTER (WHERE recu_le - message.fin > INTERVAL 1 DAY) AS en_retard,
       MAX(recu_le - message.fin) AS pire_retard
FROM read_json('sources/telemetrie/*.jsonl', union_by_name = true);
```

La météo

3.22 Mesurer les volumes et la période.

```sql
-- Attendu : 11 stations, 17 061 jours-stations sur 1 551 jours, 37 068 heures à Bagotville,
-- du 2022-01-01 00:00 au 2026-03-31 23:00.
SELECT (SELECT COUNT(*) FROM 'sources/meteo/stations.csv') AS stations,
       (SELECT COUNT(*) FROM 'sources/meteo/quotidien.csv') AS jours_stations,
       (SELECT COUNT(DISTINCT LOCAL_DATE) FROM 'sources/meteo/quotidien.csv') AS jours,
       (SELECT COUNT(*) FROM 'sources/meteo/horaire_7060400.csv') AS heures,
       (SELECT MIN(LOCAL_DATE) FROM 'sources/meteo/horaire_7060400.csv') AS premiere_heure,
       (SELECT MAX(LOCAL_DATE) FROM 'sources/meteo/horaire_7060400.csv') AS derniere_heure;
```

3.23 Compter les jours sans température, par station.

```sql
-- Attendu : 11 lignes, de BAGOTVILLE A (2 jours) à CHARLOTTETOWN A (67).
SELECT STATION_NAME, COUNT(*) FILTER (WHERE MEAN_TEMPERATURE IS NULL) AS jours_sans_temperature
FROM 'sources/meteo/quotidien.csv'
GROUP BY ALL
ORDER BY jours_sans_temperature;
```

3.24 Compter les valeurs présentes dans l'horaire.

```sql
-- Attendu : 37 068 heures ; précipitation 0 ; heure UTC 37 044 ; température 37 067.
SELECT COUNT(*) AS heures, COUNT(PRECIP_AMOUNT) AS precipitation,
       COUNT(UTC_DATE) AS heure_utc, COUNT(TEMP) AS temperature
FROM 'sources/meteo/horaire_7060400.csv';
```

Le corpus

3.25 Vérifier que chaque document du catalogue est sur le disque.

```sql
-- Attendu : 33 au catalogue, 33 présents.
SELECT COUNT(*) AS au_catalogue, COUNT(f.chemin) AS presents
FROM 'sources/corpus/index.csv' i
LEFT JOIN (SELECT substr(replace(filename, '\', '/'), 16) AS chemin
           FROM read_blob('sources/corpus/**')) f USING (chemin);
```

3.26 Répartir les documents par type.

© 2026 iAlexMG. Tous droits réservés.

```sql
-- Attendu : 5 lignes. BULLETIN 5, GUIDE 7, MANUEL 9, POLITIQUE 2 (dont 1 qui en remplace un autre),
-- PROCEDURE 10.
SELECT type_document, COUNT(*) AS documents, COUNT(remplace) AS remplacent_un_autre
FROM 'sources/corpus/index.csv'
GROUP BY ALL
ORDER BY type_document;
```

Les liens entre les sources

3.27 Suivre le numéro de série du véhicule de l'ERP à la télémétrie et aux garanties.

```sql
-- Attendu : 27 775 à l'ERP ; 6 759 en télémétrie, 0 hors ERP ; 26 732 aux garanties, 0 hors ERP.
WITH erp AS (SELECT DISTINCT SERNR AS sernr
             FROM read_csv('sources/erp/extraits/*/ZSERIE.csv', all_varchar = true)),
tele AS (SELECT DISTINCT message.sernr AS sernr
         FROM read_json('sources/telemetrie/*.jsonl', union_by_name = true)),
gar AS (SELECT DISTINCT sernr FROM 'sources/garanties/*/enregistrements.csv')
SELECT (SELECT COUNT(*) FROM erp) AS series_erp,
       (SELECT COUNT(*) FROM tele) AS series_telemetrie,
       (SELECT COUNT(*) FROM tele ANTI JOIN erp USING (sernr)) AS telemetrie_hors_erp,
       (SELECT COUNT(*) FROM gar) AS series_garanties,
       (SELECT COUNT(*) FROM gar ANTI JOIN erp USING (sernr)) AS garanties_hors_erp;
```

3.28 Suivre les concessionnaires des garanties jusqu'à l'ERP.

```sql
-- Attendu : 156 concessionnaires, 0 hors ERP.
SELECT COUNT(DISTINCT e.concessionnaire) AS concessionnaires,
       COUNT(DISTINCT e.concessionnaire) FILTER (WHERE k.KUNNR IS NULL) AS hors_erp
FROM 'sources/garanties/*/enregistrements.csv' e
LEFT JOIN (SELECT DISTINCT KUNNR FROM read_csv('sources/erp/extraits/*/KNA1.csv', all_varchar = true)) k
       ON k.KUNNR = e.concessionnaire;
```

Étape 4 — Mesurer ce que coûte un type deviné

Théorie : `Theorie_01_DE.md`, annexe A.4.

Chaque piège se montre en deux temps : la requête qui se trompe, puis celle qui corrige. Les résultats faux sont voulus. Ils ne donnent aucune erreur, et c'est ce qui les rend dangereux ; seule la jointure de 4.9 plante.

4.1 Voir les types devinés de `VBAK`.

© 2026 iAlexMG. Tous droits réservés.

```
-- Attendu : 12 lignes. MANDT BIGINT, VBELN VARCHAR, ERDAT BIGINT, ERZET VARCHAR, AEDAT VARCHAR,
-- AUART VARCHAR, VKORG BIGINT, VTWEG BIGINT, KUNNR VARCHAR, VDATU BIGINT, NETWR DOUBLE, WAERK VARCHAR.
DESCRIBE SELECT * FROM 'sources/erp/extraits/*/VBAK.csv';
```

4.2 Ajouter un jour à une date lue en entier.

```
-- Attendu : 20251231 et 20251232, une date qui n'existe pas.
SELECT MAX(ERDAT) AS derniere_creation, MAX(ERDAT) + 1 AS lendemain
FROM 'sources/erp/extraits/*/VBAK.csv';
```

4.3 Refaire le calcul correctement : tout lire en texte, puis convertir.

```
-- Attendu : 2026-01-01.
SELECT MAX(STRPTIME(ERDAT, '%Y%m%d'))::DATE + 1 AS lendemain
FROM read_csv('sources/erp/extraits/*/VBAK.csv', all_varchar = true);
```

4.4 Lire l'heure de réception d'un message telle que DuckDB la devine.

```
-- Attendu : 2025-12-15 05:25:22, de type TIMESTAMP.
SELECT recu_le, TYPEOF(recu_le) AS type
FROM 'sources/telemetrie/sorties_2025-12-15.jsonl'
LIMIT 1;
```

4.5 La lire telle qu'elle est écrite dans le fichier.

```
-- Attendu : 2025-12-15T00:25:22-05:00, de type VARCHAR. Le -05:00 a disparu en 4.4.
SELECT recu_le, TYPEOF(recu_le) AS type
FROM read_json('sources/telemetrie/sorties_2025-12-15.jsonl', columns = {'recu_le': 'VARCHAR'})
LIMIT 1;
```

4.6 Lire toute l'archive de télémétrie d'un seul motif, sans `union_by_name`.

```
-- Attendu : une seule ligne, la colonne json, de type JSON.
DESCRIBE SELECT * FROM read_json('sources/telemetrie/*.jsonl');
```

4.7 Chercher la période de la GMAO sur des dates lues en texte.

```
-- Attendu : 01/02/2024 11:00 et 31/12/2024 08:02. Faux : les bons vont de 2022 à 2025.
SELECT MIN(ouvert_le) AS premier, MAX(ouvert_le) AS dernier
FROM 'sources/usine/gmao/bons_travail.csv';
```

4.8 La chercher sur les dates converties.

```
-- Attendu : 2022-01-06 15:50:00 et 2025-12-29 11:42:00.
SELECT MIN(STRPTIME(ouvert_le, '%d/%m/%Y %H:%M')) AS premier,
```


© 2026 iAlexMG. Tous droits réservés.

```
       MAX(STRPTIME(ouvert_le, '%d/%m/%Y %H:%M')) AS dernier
FROM 'sources/usine/gmao/bons_travail.csv';
```

4.9 Erreur voulue : joindre l'horaire de la météo à ses stations, sur des clés devinées.

```
-- Attendu : une erreur. Conversion Error: Could not convert string '502S001' to INT64.
SELECT s.STATION_NAME, COUNT(*) AS heures
FROM 'sources/meteo/horaire_7060400.csv' h
JOIN 'sources/meteo/stations.csv' s USING (CLIMATE_IDENTIFIER)
GROUP BY ALL;
```

À retenir — une clé de jointure a le même type des deux côtés, et c'est à vous de le déclarer. DuckDB a deviné `CLIMATE_IDENTIFIER` entier dans l'horaire (`7060400`) et texte dans les stations (`502S001`) : la jointure plante. Ce piège-là, au moins, se voit. Devant un `Conversion Error` sur une jointure, on cherche d'abord une clé devinée de deux façons.

4.10 Refaire la jointure en déclarant le type de la clé.

```
-- Attendu : BAGOTVILLE A, 37 068 heures.
SELECT s.STATION_NAME, COUNT(*) AS heures
FROM read_csv('sources/meteo/horaire_7060400.csv', types = {'CLIMATE_IDENTIFIER': 'VARCHAR'}) h
JOIN 'sources/meteo/stations.csv' s USING (CLIMATE_IDENTIFIER)
GROUP BY ALL;
```

Étape 5 — Sauvegarder les requêtes dans le dépôt

Théorie : `Theorie_01_DE.md`, annexe A.5.

Le notebook vit dans votre profil Windows, hors de `01_de`. Ouvrir une deuxième Anaconda Prompt (la première fait tourner l'interface), activer `formations`, se placer dans `Projet_Boreal_intermediaire` :

```
mkdir 01_de\sql
notepad 01_de\sql\premier_regard.sql
```

Accepter la création. Coller les requêtes du notebook, cellule par cellule, commentaires compris, chacune finie par son `;`. Mettre la requête 4.9 en commentaire (un `--` devant chaque ligne) : elle plante exprès, et `duckdb -f` s'arrête à la première erreur. Enregistrer, fermer.

Rejouer le fichier d'un trait, pour prouver qu'il est complet :

```
duckdb -f 01_de/sql/premier_regard.sql
```

Attendu : 40 résultats défilent sans erreur ; le dernier est la ligne `BAGOTVILLE A`, 37 068 heures.

Valider le fichier :

© 2026 iAlexMG. Tous droits réservés.

```
git -C 01_de add sql\premier_regard.sql
git -C 01_de commit -m "Cours 01 : les requêtes du premier regard"
git -C 01_de status
REM Attendu : nothing to commit, working tree clean.
```

Pour tout fermer : `.exit` à l'invite `D` de la première Anaconda Prompt, puis fermer l'onglet.

Dépannage

Symptôme	**Correctif**
`No files found that match the pattern`	`duckdb -ui` n'a pas été lancé dans `Projet_Boreal_intermediaire` : `.exit`, se placer dans `Projet_Boreal_intermediaire`, relancer `duckdb -ui`
La cellule n'affiche qu'un seul résultat	elle contient plusieurs requêtes : une requête par cellule
La page ne répond plus	l'Anaconda Prompt a été fermé : relancer `duckdb -ui` depuis `Projet_Boreal_intermediaire`
L'extension `ui` ne s'installe pas	pas d'Internet au premier lancement : se connecter, puis relancer
2.4 trouve des fichiers absents ou altérés	une source a été modifiée à la main : relancer le générateur (dernière ligne)
`Binder Error: Referenced table "message" not found!`	l'archive de télémétrie a été lue sans `union_by_name = true` (voir 4.6)
`duckdb -f` ne trouve pas `premier_regard.sql`	le Bloc-notes l'a enregistré en `premier_regard.sql.txt` (`dir 01_de\sql` le montre) : `ren 01_de\sql\premier_regard.sql.txt premier_regard.sql`
`duckdb -f` s'arrête sur `Conversion Error`	la requête 4.9 n'a pas été mise en commentaire
`duckdb -f` s'arrête sur une autre erreur	une requête mal collée, ou sans son `;` : corriger le fichier dans le Bloc-notes, relancer
Un compte ne donne pas le chiffre attendu	relancer le générateur, depuis `Data_Domain` : `python _univers/boreal/generer_boreal.py --projet Projet_Boreal_intermediaire`

© 2026 iAlexMG. Tous droits réservés.